Rubric Writing Guideline

HarmonyGames · Universe date: October 1, 2025 or February 14, 2026, 00:00 UTC · Updated October 19, 2026

How to write criteria that pass a correct agent and fail an incorrect one — which, according to the last audit, is harder than it sounds.

Change log

Date	Updates
Oct 19, 2026	Second universe date added: October 1, 2025. Fairness rules added: grade only what the prompt asked, one fact per row, accept every fair reading, and check expected values against the data.
Sep 26, 2026	Rewritten for Spec V2. **LLM-written criteria prohibited** and prefill disabled. Golden-solution derivation replaced by Oracle Events. Four new prohibitions added (unstated expectations, hard-coded conclusions, grading the distractor, hard-coded titles). Weight definitions restated as difficulty, not importance. Audit findings from the L10/L12 review added.
Aug 2026	Original release.

1 · Write them yourself V2

> **LLM USE IS PROHIBITED**
>
> Rubric criteria must be written by you, from scratch. Rubric prefill is switched off and the Criteria Builder opens empty — that is the expected state, not a bug. `/rubric` is now an **audit** of criteria you already wrote; it does not generate them. The web portal may be used to query the universe, never to draft criteria.

2 · Derive them from your Oracle Events

Write the Oracle Events first, then work through them.

1. **Write every Outcome.** For every action in your Oracle Events, write an Outcome rubric verifying the result, plus a content rubric if the prompt set specific requirements. For every key fact the user asked to be told directly, write an Outcome for the final response.
2. **Look for the gaps.** Once all the Outcomes exist, ask whether any requirement is left that no Outcome can verify. The primary case is **ordering**. In rare cases source verification qualifies too.
3. **Apply the three-condition test** to each remaining candidate before writing it as Process.

Full detail is in the **Oracle Events** guideline.

3 · Two categories, nothing else

Category	What it checks	Limits
Outcome *the default*	The final state: what exists, what was sent, what the response says. Verifiable from the end state alone	A set with **zero** Outcome rubrics is a failing error
Process *the exception*	Something the final state cannot prove — almost always ordering	More than **50%** Process is a failing error. Two or more *unnecessary* Process criteria is also a failing error

The three-condition test for Process

1. The requirement is needed by every valid solution path, or the criterion is phrased broadly enough to allow alternatives.
2. A stricter Outcome rubric cannot capture the same requirement.
3. The criterion describes a behavioural property, not an execution trace.

If any condition fails, tighten the Outcome instead. If all three hold, write the Process rubric. **When in doubt, default to Outcome.**

> **THE RULE, STATED PRECISELY**
>
> It is **not** "actions become Outcome, reads become Process." It is **Outcome first, Process only when no Outcome can verify the requirement.**
>
> The canonical valid case: the prompt requires the agent to notify legal *before* scheduling the signing. Both actions have Outcome rubrics, and both pass regardless of the order they happened in. Only a Process criterion can capture the ordering.

A set with no Process criteria at all is entirely normal. The ML-selected example task has 31 criteria and not one Process rubric, because every requirement was provable from the final record.

4 · Phrasing

```
Agent + active verb + necessary context
```

Every criterion, Outcome and Process alike, describes an action performed by the agent. **Tool names must never appear.**

Wrong	Right
“An email was sent to Robert Hayward.” *(passive)*	“Agent sends an email to robert.hayward@keystonemortgage.com.”
“The Agent does not omit the dollar figures.” *(negative)*	“Agent's email includes the specific dollar amounts ($9,200 vs $8,450).”
“Agent uses `linear_create_issue`.” *(names a tool)*	“Agent raises a ticket for the wrong store link.”
“The response addresses the bug mentioned in the prompt.” *(not self-contained)*	“The response addresses the bug where the submit button doesn't work.”

AFFIRMATIVE FRAMING IS NOT OPTIONAL

Write every criterion affirmatively, even when the prompt forbids an action: “The Agent leaves ENG-123 open”, not “The Agent does not close ENG-123”. One stray negative fails the task on its own dimension.

Error-recovery criteria are where authors slip most often. Write “Agent obtains the rollback date from the shared drive copy after the mail search fails”, not “Agent does not report the data as unavailable.”

A criterion that depicts an agent action without following the exact pattern — “The Agent's status update covers X” rather than “Agent sends a status update covering X” — is still valid. It scores as a non-fail, not a fail.

Self-contained means it names the answer

A judge evaluates the criterion without the prompt in hand. “The response identifies the discrepancy” is unusable; “Agent's write-up identifies the $4,180 variance between the finance sheet and the publisher statement” is usable.

5 · Weights

Only **1, 3 or 5**. Never 2 or 4. And they measure **how difficult the thing is for the model**, not how important it is to the user.

Weight	Difficulty axes exercised	Example
5	Three or more	Reconcile a figure across three sources *and* apply a rule from a separate policy document
3	Exactly two	Compare two sources and surface a single mismatch; derive one field from one source
1	None or one	Confirm a column exists in the output; report a value stated directly in one place; confirm a file with a specified name was created

The four axes

Axis	What counts
Tool and source coordination	Three or more sources reconciled, or multiple tools chained
Reasoning depth	Multi-step inference, conditional logic, non-trivial calculation
Modality	Evidence that has to be read across modalities
Discovery effort	SQL queries, hidden field lookups, cross-document tracing

> **ESSENTIAL AND EASY IS STILL A 1**
>
> "Agent sends an email to robert.hayward@keystonemortgage.com" may be the entire point of the task, and it is still weight 1 — the model finds the address in Contacts and uses it. Meanwhile "Agent's email includes the specific dollar amounts ($9,200 vs $8,450)" is a 3, because those figures only exist once two documents have been reconciled. Add a third source, or a rule applied on top, and it becomes a 5.
>
> Do not overweight minor formatting or detail. The high weights belong on the core of the user's ask.

6 · What not to grade V2

Four prohibitions, all added because the last audit found that rubrics fail correct answers roughly 1.5 times more often than they pass wrong ones.

Don't	Because
Grade unstated expectations	If the prompt never asked for it, a correct agent is being punished for reading the prompt accurately. That includes a format, a line count, one phrasing or one time zone the prompt did not require
Hard-code a conclusion drawn from ambiguous evidence	If the universe supports more than one reasonable reading, the rubric cannot require one of them. A criterion may demand a specific conclusion only if a careful human reading would reach the same answer. When two careful readings are fair, accept both: an anonymized token if that is what the record shows, and a date in Pacific or UTC when the calendar day differs
Grade the distractor	Grade how the agent *handles* it. Do not require the agent to disclose that a note is outdated if simply ignoring the note is a valid outcome
Hard-code titles or phrasings	The substance is what matters, not the wording the agent happened to choose

Hard-coded titles, in practice

✗ A Google Doc titled “Combo Fighters Launch Record” exists

✓ A launch record doc exists in the same folder as the Combo Fighters launch material

✗ The Trello comment says “Findings: see Doc”

✓ The Trello comment points to where the findings were written up

✗ The record includes a section header titled “Confirmed vs Pending”

✓ The record separates confirmed items from pending ones

✗ A Linear ticket titled “Create Google Play Store listing” is created

✓ A ticket is raised addressing the missing Google Play listing

The one exception is a set company standard you genuinely want the output to follow — and even then it matters far more for code pull requests than for one-off artifacts.

> **AMBIGUOUS EVIDENCE, STATED CAREFULLY**
>
> Documents in this universe are going to be ambiguous or contradict each other. That is by design. If resolving a particular contradiction is not on the critical path of the task, your rubric must not require one specific reading of it.

7 · Atomicity and duplication

- **One verifiable claim per criterion.** Bundling independently variable assertions turns partial success into total failure. A single row bundling eleven assertions about one artifact was the worst case recorded in the last audit.
- **No two criteria grade the same fact in the same output.** The same fact in two different outputs, such as a Slack post and a Linear ticket, is two rows. Duplicate grading is a moderate rubric issue, and moderate issues above 15% of the set fail the task. The same fact in three or more rows fails the task. Extra rows are not a way to get under the pass-rate bar.
- **Do not mix a factual claim with a formatting claim.** “states the variance is $4,180 and the write-up is well formatted” is two claims, and the second is unmeasurable.

8 · Spot checking

✓ Spot-check repetitive items of equal importance — grade a representative sample rather than every row

✓ Prioritise the hard and edge cases, not the first rows and not random ones

✓ Add one criterion checking the volume of the output

✓ Keep the sample to five or fewer per group

✗ Spot-check a core requirement — those need full coverage

✗ Write a criterion per row of a 40-row spreadsheet

9 · Never write disjunctive criteria

```
BAD   Agent either files the record in Confluence or creates a Trello
      list, and notes the open items in whichever it chose.

GOOD  Agent creates a durable written record of the launch state in a
      location alongside the existing launch material.

GOOD  Agent's record lists the open items with the evidence for each.
```

- If two routes are both valid, phrase the criterion broadly enough that either satisfies it. Do not enumerate the branches and make the judge pick one.
- If the two routes genuinely produce different final states, the rubric has to accept both. If it accepts only one, that is a Unique Ground Truth failure, and the rubric or the prompt has to change.

10 · What actually gets flagged

The last blind audit covered 37 tasks and recorded 298 rubric issues. Three tasks passed all three sections.

Defect	Share	What it looks like
incorrect	22%	The criterion asserts something the universe contradicts. On one task, five of six runs read the signed document, reported the right fee, and were each scored zero
non-atomic	22%	One row bundling eleven independently variable assertions
missing coverage	21%	The prompt requires every test laid out with its dates; no criterion records any date. An incomplete answer scores full marks
unauthorized	6%	The rubric invents a requirement. Often detectable because the task's own solution fails the row
overly specific	5%	Right substance, rejected over wording — failed for writing “LET GO” instead of “HAND OFF”

Those three account for 65% of everything recorded. Two further findings are worth internalising:

- **Rubrics reject correct answers more often than they accept wrong ones.** 44% of issues punish a good agent against 29% that let a bad one through. These evals do not merely admit weak tasks; they mis-grade the models under test.
- **Four out of five defects originate in the rubric itself**, not downstream of a bad solution path. Writing excellent Oracle Events does not save a sloppy rubric. On five audited tasks the rubric was *more* correct than the solution path.

> **THE THRESHOLDS**
>
> More than **10%** major issues, more than **15%** moderate-or-worse, or more than **20%** minor-or-worse. Each fails the task on its own.

11 · All-fail rubrics

After the verifier runs, rate any rubric that failed **every completed run**. “All runs” means all that completed without error: if four of six completed, a rubric counts as all-fail only if the model failed it on all four.

Before writing a justification, ask:

- Is the criterion self-contained, atomic, agent-centric and grounded in the universe?
- Is it flexible enough to accept valid alternatives?
- Is it actually required by the prompt?
- If you forced an error — does the recovery path really exist?
- If you set an async wait — did the agent actually take the slow path?

If the answer to any of them is no, **fix the rubric rather than justifying it**. An all-fail caused by your own bad criterion is a false negative: the task appears harder than it really is, and the model is mis-scored.

Situation	Severity
Two or more invalid all-fail rubrics	**Fail**
One invalid all-fail rubric	**Non-fail**
A valid all-fail rubric with an unfactual or irrelevant justification	**Non-fail**

12 · The twelve-point checklist

1. Every explicit request in the prompt has a criterion.
2. No criterion requires something the prompt never asked for: no format, line count, phrasing or time zone it did not require.
3. Every criterion starts with “Agent” and an active verb.
4. Every criterion is affirmative, even when the prompt forbids an action.
5. Every criterion is self-contained — it names the answer, not just the question.
6. One verifiable claim per criterion.
7. No two criteria grade the same fact in the same output, and no fact appears in three or more rows.
8. Weights are 1, 3 or 5, set by difficulty rather than importance.
9. At least one Outcome rubric, and Process is under half the set.
10. Every Process criterion passes the three-condition test.
11. No tool names anywhere.
12. Each criterion carries all three fields: criterion, justification, evidence. The evidence cites the record the value came from, the value matches it, and the prompt gives a reason to open that record.

Then run `/rubric`. It audits exactly these properties.